

Exercise 1 : A data structure for select and rank

(a)

```
public class RankSelect{

    private ArrayList<Integer> number_of_true;          //in the field of our data structure , we have the number of true array , the element is the rank(A,i) in i position

    public RankSelect( ArrayList<Boolean> A ){          //this is the constructor , initializes the data structure

        number_of_true = new ArrayList<>();

        int count_of_one = 0;

        for( int i = 0 ; i < A.size() ; i++ ){

            number_of_true.add(count_of_one);

            if( A.get(i) )
                count_of_one++;

        }

        number_of_true.add(count_of_one);

    }

    public int rank(int i){

        assert ( i >= 0 ) && ( i < number_of_true.size() ) : "precondition , since the size of A = the size of number_of_true , so 0 <= i < n = size of A "

        return number_of_true.get(i);

    }

}
```

COMMENT : 1. In the initializes part of our data structure (i.e the constructor part) , the Worst time complexity is $O(n)$, because we have only one for loop in size n

 2. in the rank(int i) , the Worst time complexity is $O(1)$, because all code in this method take $O(1)$.

=====

(b)

proof:

```
public int select( int j ){
    ...
}
```

in Exercise (a) , we know that the method rank(int i) is to return the number of "true" in the array $A[0,1,\dots,i-1]$

and we note that the outputs of the method rank(int i) actually are monotonic non-decreasing

so , actually , select(j) is the smallest index k such that $\text{rank}(k + 1) == j + 1$.

that is , we want to find that k .

More , since the output of the rank operation is monotonic non-decreasing ,

so we can do binary search over the possible index range $[0, n-1]$ to find that k .

we know that the Worst case of the binary search is $O(\log n)$, the time complexity of rank(int i) is $O(1)$. So the total Worst time complexity is $O(\log n)$. DONE!

Therefore , we proved that just by using rank operation , and without storing additional information in the data strucure , we can implement the select(int j) operaiton.

Q.E.D.

Exercise 2 : Multi-way sorting

(a)
The Worst case time complexity of this algorithm is $O(m \cdot n^2)$.
proof :

```
the structure of the algorithm is like this : (just pseudo code):  
  
According to the proposed algorithm, B is initially initialized to A[0], so its starting length is m.  
  
public ArrayList<Integer> ThisAlgorithm(){  
    for ( int i = 1 ; i < n ; i++ ){  
        B = MERGE(B, A[i])    // Merge B and A[i]  
                             // Update B equal to the previous one  
    }  
    return B;  
}
```

Since Merge(M1 , M2) will takes $O(|M1| + |M2|)$
So , This algorithm will takes $O((m+m) + (2m+m) + (3m+m) + \dots + ((n-1)m+m))$
 $= O(2m + 3m + 4m + \dots + nm)$
 $= O(m * (2 + 3 + 4 + \dots + n))$
 $= O(nm + m \cdot n^2) = O((n+n^2)m) = O(m \cdot n^2)$

Therefore , the time complexity of this algorithm is $O(m \cdot n^2)$. (actually , for this time algorithm , Best time complexity = Worst time complexity)
Q.E.D.

=====

(b)
We will use heap to optimize our algorithm.

By hypothesis , we have that :
A[0] is sorted increasingly
A[1] is sorted increasingly
...
A[n-1] is sorted increasingly

Our idea is :
we will make a Min-Heap of maximum size n .

Crucially, to remember which array an element came from,

the elements in the heap must be objects storing three values:(value, arrayIndex, elementIndex).

The heap will be ordered by the value.

since in the heap , delete takes $O(\log n)$ times , insertion take $O(\log n)$ times

so in this heap , we will delete the top element(that is the smallest element in the heap) and put it into our result array , then insert the next element in some $A[i]$,

since we have total nm elements,

Therefore , the time complexity of this algorithm is $O(mn \cdot \log n)$.

Our pseudo code :

```
public ArrayList<Integer> OptimizeAlgorithm(){
    MinHeap Heap = new MinHeap();
    for (int i = 0; i < n; i++) {
        Heap.insert( (A[i][0], i, 0) ); // Insert: (value, arrayIndex, elementIndex)
    }
    // This step takes  $O(n)$  times.
    ArrayList<Integer> ResultArray = new ArrayList<>();
    while ( !Heap.isEmpty() ) {
        Tuple minItem = Heap.extractMin(); // Pop the smallest tuple
        ResultArray.add(minItem.value);
        // Now we use the saved indexes to find the exact next element!
        if (minItem.elementIndex + 1 < m) {
            int nextVal = A[minItem.arrayIndex][minItem.elementIndex + 1];
            Heap.insert( (nextVal, minItem.arrayIndex, minItem.elementIndex + 1) );
        }
    }
    return ResultArray;
}
```

Therefore , followed by the optimize algorithm , the time complexity is $O(mn \cdot \log n)$.

Exercise 3 : A data structure for anagram checking

Our idea is :

Why we don't use a HashMap do achieve this :

To achieve a strict worst-case time complexity of $O(m)$,
we cannot use a HashMap because hash collisions can degrade its worst-case time to $O(N)$.

Instead, we will use a Trie (Prefix Tree) :

We will use the "id card" idea. Since the alphabet has only 26 letters,
we can use Counting Sort to sort any string w into a canonical string w' in $O(m)$ time , that is , the `id_card` is always sorted in the English alphabet order.
e.g. "cba" -> its id card is "abc"
e.g. "silent" -> its id card is "eilnst"

We will insert this sorted "id card" into the Trie.

This is the design of the data structure : (pseudo code)

```
// Define a Trie node that also stores the count of anagrams
```

```
class TrieNode {
    TrieNode[] children = new TrieNode[26];
    int count = 0;
    //this is to count how many words have the same id_card (i.e. how many anagrams of this id_card we have inserted into the Trie)
}
```

```
public class StringSet {
    TrieNode root;

    public StringSet() {
        root = new TrieNode();
    }
}
```

```
void insert(String w){

    // Step 1 : Compute the sorted id_card of w.

    String id_card = compute_sorted_id_card(w);

    // Step 2 : Walk down the Trie.

    TrieNode curr = root;

    for (char c : id_card.toCharArray()) {

        if (curr.children[c - 'a'] == null){
            curr.children[c - 'a'] = new TrieNode();
        }

        curr = curr.children[c - 'a'];

    }

    curr.count++;
    // Mark the end of the id_card

}
```

```
//This is the auxiliary method to compute the sorted id_card of a string w.
private String compute_sorted_id_card(String w) {
```

```
    int[] freq = new int[26];

    for (char c : w.toCharArray()) {

        freq[c - 'a']++;

    }

    String id_card = "";

    for (int i = 0; i < 26; i++) {

        for (int j = 0; j < freq[i]; j++) {
```

```

        id_card += (char)(i + 'a');
    }
}

return id_card;
}

int countAnagrams(String w){
    // Step 1 : Compute the sorted id_card of w.
    String id_card = compute_sorted_id_card(w);

    // Step 2 : Walk down the Trie.
    TrieNode curr = root;
    for (char c : id_card.toCharArray()) {
        if (curr.children[c - 'a'] == null){
            return 0;
            // id_card not found
        }

        curr = curr.children[c - 'a'];
    }

    return curr.count;
}
}

```

SOME COMMENT ABOUT THE CODE :

1. Why compute_sorted_id_card(w) takes $O(m)$ time?

Instead of using a general sorting algorithm which would take $O(m \log m)$ time,

we use Counting Sort. We count the frequencies of characters in a fixed-size array of 26.

The counting step takes $O(m)$ time, and rebuilding the string takes $O(26) = O(1)$ time. Thus, the worst-case time complexity is strictly $O(m)$.

2. About the time complexity , why this two methods take $O(m)$ time ?

The insert method :

The compute_sorted_id_card(w) takes $O(m)$ time, as explained above.

Walking down the Trie also takes $O(m)$ time, since we process each character of the id_card once.

Therefore, the overall time complexity of the insert method is $O(m)$.

The countAnagrams method :

Similar to insert, the compute_sorted_id_card(w) takes $O(m)$ time.

Walking down the Trie also takes $O(m)$ time.

Therefore, the overall time complexity of the countAnagrams method is $O(m)$.

Exercise 4: A data structure for ranking players

Our idea is :

We will use AVL tree to solve this problem.

The node of the tree has three variables:

int score : the score of this game

int count : how many players have this score at this moment

int total : the sum of the nodes of the subtree of this node

We will also use an array score[n] to remember the score of each players.

For example , suppose we have 6 players , now they all have 0 points.

so , now :

our AVL tree is like this :

[score:0 , count:6 , total:6]

our array score[6] is like this :

[0 , 0 , 0 , 0 , 0 , 0]

now , we add 1 point to players zero , then :

our AVL tree is like this :

[score:0 , count:5 , total:6]

 \
 \
[score:1 , count:1 , total:1]

our array score[6] is like this:

[1 , 0 , 0 , 0 , 0 , 0]

Our pseudo code is :

```
class Node<Integer>{
    int score;
    int count;
    int total;
    Node<Integer> left = null;
    Node<Integer> right = null;
}
```

```
public class Ranking( int n ){
```

```

Node<Integer> root;    //our AVL tree

int score[n];          //our array score[n]

public Ranking(){

    //initialize the array score[n]
    for(int i = 0 ; i < n ; i++){
        score[i] = 0;
    }

    //initialize the AVL tree
    root.score = 0;
    root.count = n;
    root.total = n;
}

void score(int i){

    assert (i >= 0 && i < n) : "Precondition";

    //update the AVL tree

    in AVL tree , we search the node "Node" that have score score[i] , this step take O(log n) in AVL tree

    Node.count--; (if count go to zero , then delete this node)

    if(Node.count == 0){
        delete(Node);
        and do update rotation operation for AVL tree;
    }
    else{
        update total upward(Node).          //total-- for all parent of Node
    }

    now , we seach the node "Next" that have score ( score[i] + 1 ), this step also take O(log n) time

    if(next != null){
        next.count++;
        update total upward(Next);          //total++ for all parent of Next
    }

    else{
        inset the node "Next"
    }

    //update the array score[n]

    score[i]++;

}

int ranking(int i){

    assert (i >= 0 && i < n) : "Precondition";

    in AVL tree , we search the node "Node" that have score[i] (that is the player i 's score) , this step take O(log n) in AVL tree
    During the O(log n) search path for score[i], we maintain a sum 'k' (initially 0) to count all players with strictly greater scores.
    At each step of the search from the root:
    - If target score > current node's score: we go RIGHT, do nothing to k.
    - If target score < current node's score: we go LEFT.

```

Since all players in the right subtree and the current node have higher scores,
we update: $k = k + \text{current.count} + \text{current.right.total}$.

When we finally find the "Node" with $\text{score}[i]$, the final ranking is:
 $\text{return } k + \text{Node.count} + \text{Node.right.total}$;

```
    }  
}
```

COMMENT : 1. In the void $\text{score}(\text{int } i)$, update the array $\text{score}[n]$ just take $O(1)$ time
 update the AVL tree take $O(\log n)$ time in the worst case
 rotation operation take $O(\log n)$ time (we do it for all parent of Node)

So the total time complexity in the Worst case is $O(\log n)$.

2. In the int $\text{ranking}(\text{int } i)$, similiary it takes $O(\log n)$ in the worst case.